ReactiveDynamics.jl Modeling Copilot

A guided workflow for decisions whose answer depends on how an operation behaves over time: shared capacity, lead times, feedback, decisions taken mid-course, the dynamics a spreadsheet flattens.

Describe the decision and how the business runs. The copilot builds a high-fidelity model beneath the conversation, tries to break it, and shows what drives the answer.

Decisions of this kind — a launch-capacity reservation, a bid under a staffing ceiling, a portfolio whose funding follows its own results, a service backlog that feeds back into demand — today need a modeler, weeks of work, and trust in a workbook nobody else can read. The copilot turns that into a guided session with the person who owns the decision, with each step checked by the engine rather than by the reader's faith. The conversation stays in business terms, never the model's mechanics, and every number says what it rests on and replays exactly from its seed. This page sets out the vision, what exists today, and the direction.

what it is a reusable modeling capability: an interview, a running model of how your operation behaves, a health pass that can refuse, and a briefing that ships as a generated report — run in conversation

who drives it an analyst or a strategy lead, in business language. Today it runs in Claude Code on a checkout of the repository with Julia installed, so someone technical sets it up; the direction is a tool any assistant can call (see §5)

why it can be trusted the model is a checked data document in a closed vocabulary, never code — validated before it runs, reproducible from a seed afterwards

the case on this page a launch-supply decision in pharma, captured with AI on both sides: the operator was a role-played agent answering from a fact sheet sealed before the first message. The company is invented; the numbers are real runs of the model, on paired single-seed paths, not ensembles

---

The expensive part of a decision like this is not the arithmetic. It is getting the question right, getting the operation right, and knowing which parts of the answer you are entitled to defend in the room. That is why these questions go to a specialist, take weeks, and come back as a workbook the decision-maker has to take on faith.

The copilot does that work with the decision owner. It interviews them about the decision, turns their description of the operation into a running model, stress-tests the model against what they said the business does, and only then reports a number — with the mechanism that produced it and an honest account of the

evidence behind it. The same capability serves the next question and the one after: a way of working, not a one-off model.

1 · MORE DECISIONS GET A REAL MODEL, AND THE OWNER STAYS IN THE LOOP

The threshold for a real model drops. Today a question about how an operation behaves — who waits when the line is full, what a reservation is worth before the readout, what happens to intake when the backlog grows — gets a model only when it is big enough to justify a specialist. The rest get a workbook. A workbook can hold a monthly time series and even a Monte Carlo draw, but it cannot easily hold *one shared line allocated month by month between products whose launch dates are themselves random, plus a rule that changes the plan when a readout lands*. Building that in a spreadsheet means writing a simulator inside it that nobody can check. In the case on this page, a sheet built on average dates has no path on which a late qualification delays a filing, so it could not have shown the $72M loss on one path that made the operator change the plan.

The owner stays in the loop. The interview runs in business language with the person who owns the decision. Every assumption lands in a written brief they agree to, including which inputs were their own estimates and what they expect the answer to be, so the result is compared against the expectation rather than quietly replacing it.

The answer is defensible, not just plausible. Each number arrives with the evidence behind it, the command that produced it and the seed that replays it. When the CFO asks "what if the second line costs more", the answer is one edited input and a batch of re-runs the same day, not a rebuilt workbook.

It compounds. A study is a directory — brief, model, health report, briefing, every command — and it ships as a generated report the modeler can forward as is. The next question about the same operation starts from that model.

2 · THE ENGINE MODELS MORE THAN QUEUES; THE COPILOT'S CHECKS ARE CATCHING UP

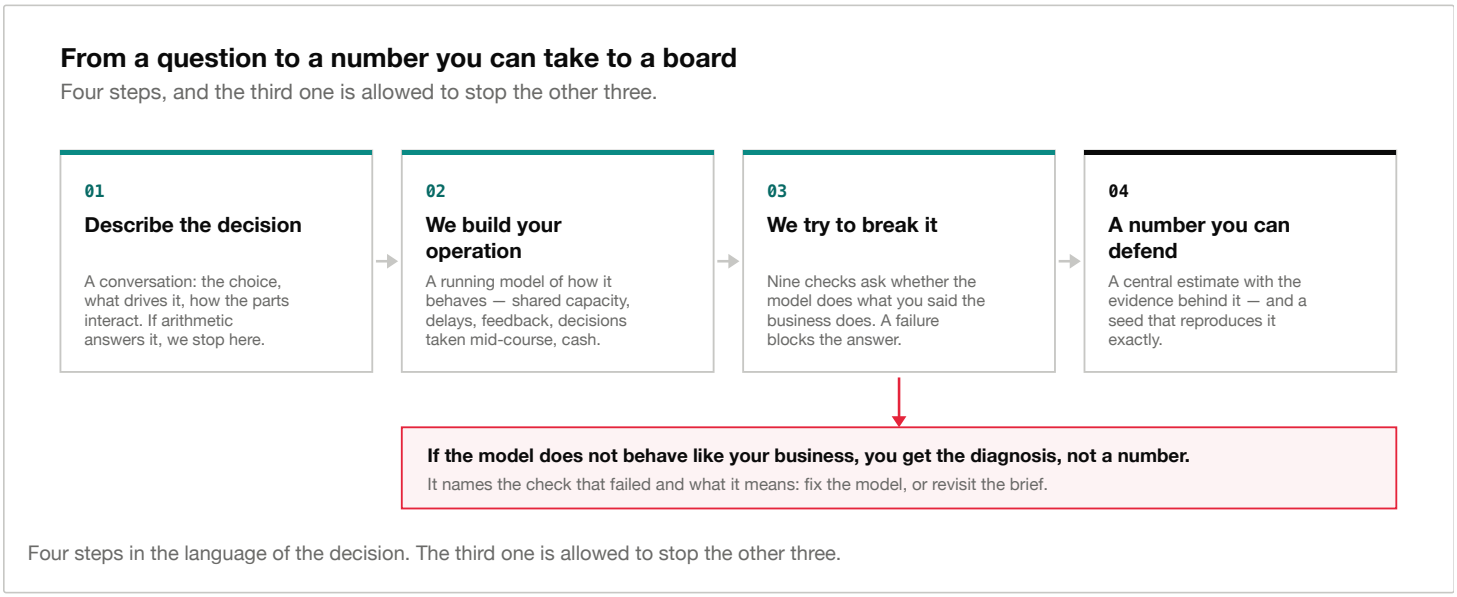
The engine underneath is a general process simulator. Contention for shared capacity is the pattern the first studies exercised, and it is where the copilot's checks are sharpest. It is one of several patterns a model can carry, often in the same model:

Pattern	Business example	Copilot today
Contention	two launches on one fill line; a bench of scientists	authored and checked end to end

Lead times and clocks	tech transfer, a build, a regulatory review	authored and checked
Feedback	a backlog that throttles intake; share lost after a stock-out; funding that follows results	authored and run — the case model has 12 rates that read the live state — but no check yet tests loop behaviour
Decisions taken mid-course	review a contract at a readout; an allocation policy	authored and run; the case turned on one
Entities with a lifecycle	projects with attributes moving through phases	authored and run
Uncertainty	readout odds, lead-time ranges	authored and run; options compared on the same random draws
Coupled models	a supply model feeding a commercial model; several sites	in the engine; not yet expressible as a copilot model document

The interview's qualifying test and the plausibility checks were written for contention. Widening them to feedback and coupled models is part of the direction in §5 (W25).

3 · FOUR STEPS, AND A CHECK THAT CAN STOP THE ANSWER



Four steps in the language of the decision. The third one is allowed to stop the other three.

Step	What happens	What you get	Gate
Frame	an interview: the decision, the real alternative, what is shared, what takes time, what reacts to what, what counts as an answer	a written brief, agreed before anything is built	the question needs a model, not arithmetic
Author	the operation becomes a running model, one version per option	a checked model document per option	the validator returns no errors

Diagnose	plausibility checks ask whether the model does what you said the business does	a health report, every failure stated	a failed check blocks the number until it is fixed, or explained in writing and accepted by the owner
Report	every option is run on the same random outcomes; differences are read against their own noise	a briefing: estimate, mechanism, and what it rests on	every number carries its evidence and a seed

If the question is arithmetic, the interview says so and stops. Nothing shared, nothing delayed, nothing reacting: a spreadsheet is the right tool. Numbers the owner is unsure of are recorded as estimates, and the published audit of each captured session says which of them were tested.

A failed check is not a footnote. In the case, every option failed the same check, that every step of the model fires at least once in a run. That was by design, not a defect: one network carries every option and every readout outcome, so any single run leaves the steps of the paths it did not take idle, such as a second line nobody built, or Tern's production when Tern's trial fails. The study reported a number only after `DOCTOR.md` showed that each idle step fires in the runs where its path is taken, and a flag that lets an option declare its intentionally idle steps is on the tracker (W20). A horizon warning also stands: both products are still ramping at end-2032, so every value is a within-horizon number, and the report says so.

Term	Meaning on this page
option / arm	one version of the model per choice on the table, identical except for what the choice changes
seed	the number that fixes every random draw, so a run replays exactly
paired paths	the options run on the same seeds, so they see the same readout dates and lead times, and the difference is the option, not the luck
ensemble	many seeds of one option, summarised with a spread; the stronger form of evidence, not used in this case
check	a plausibility test on a running model, such as whether a bottleneck really binds

4 · THE NUMBER CAN BE TRUSTED BECAUSE THE MODEL IS DATA, NOT CODE

The operation ends up as a structured document in a closed vocabulary the engine understands — places that hold resources, transitions that consume them over time, rates, rules, costs, what each option changes. Nothing in the document is a program, and nothing in it can name one. That is what makes the rest possible:

- **Checked before it runs.** A pure validator reads the document and returns every problem with its exact location. A malformed model produces diagnostics, not a plausible wrong number. That is what makes it safe to put a language model in the

authoring seat: it proposes, a deterministic checker disposes, and the loop repeats until the checker is silent.

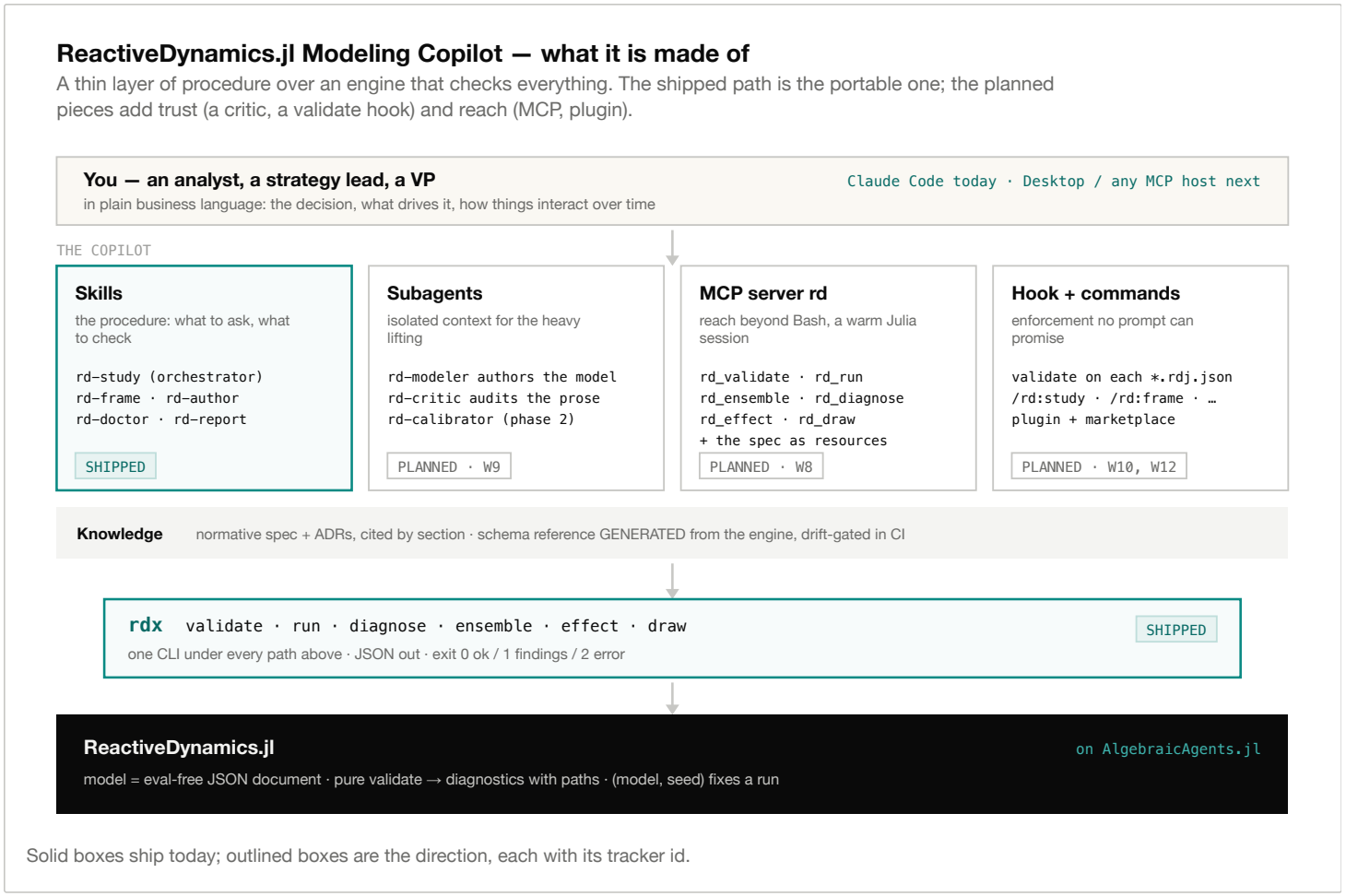
- **Checked again, for meaning.** A schema cannot tell you that your bottleneck never binds or that the horizon cuts off the payoff. The plausibility checks do.
- **Reproducible by construction.** A run is fixed by the model and a seed; options are compared on the same random outcomes.
- **Guarded in CI.** The reference the copilot reads is generated from the engine and fails the build if it drifts, and a known-answer test re-derives a published result through the same path on every change.

Where the guarantee stops today. It covers the model, not all the arithmetic around it. In the case, the documents were emitted by a short generator script, each then validated as data; and the odds-weighting behind the headline ran in analysis scripts the assistant wrote during the session. Those scripts are published with the study and were re-run independently over the stored runs, and every figure matched — but they are code, reviewed by reading. Moving that weighting into the engine's own `effect` command is part of the direction (W27).

Contrast with a code-writing assistant. An assistant using a general simulation library writes the model as code, and code cannot be checked before it runs: it crashes or returns *something*, and *something* is the dangerous case. Here the machine can author the model, be wrong, be told exactly where, and try again.

5 · INSIDE THE COPILOT: WHO DOES WHAT

The copilot is a thin layer of procedure over an open-source engine. **Skills** carry the procedure for each phase: what to ask, what to check, how to report. A single command-line tool, `rdx`, does the work: validate, run, diagnose, run ensembles, measure the effect of an option, draw the network. Underneath is ReactiveDynamics.jl, a stochastic process engine built on AlgebraicAgents.jl.

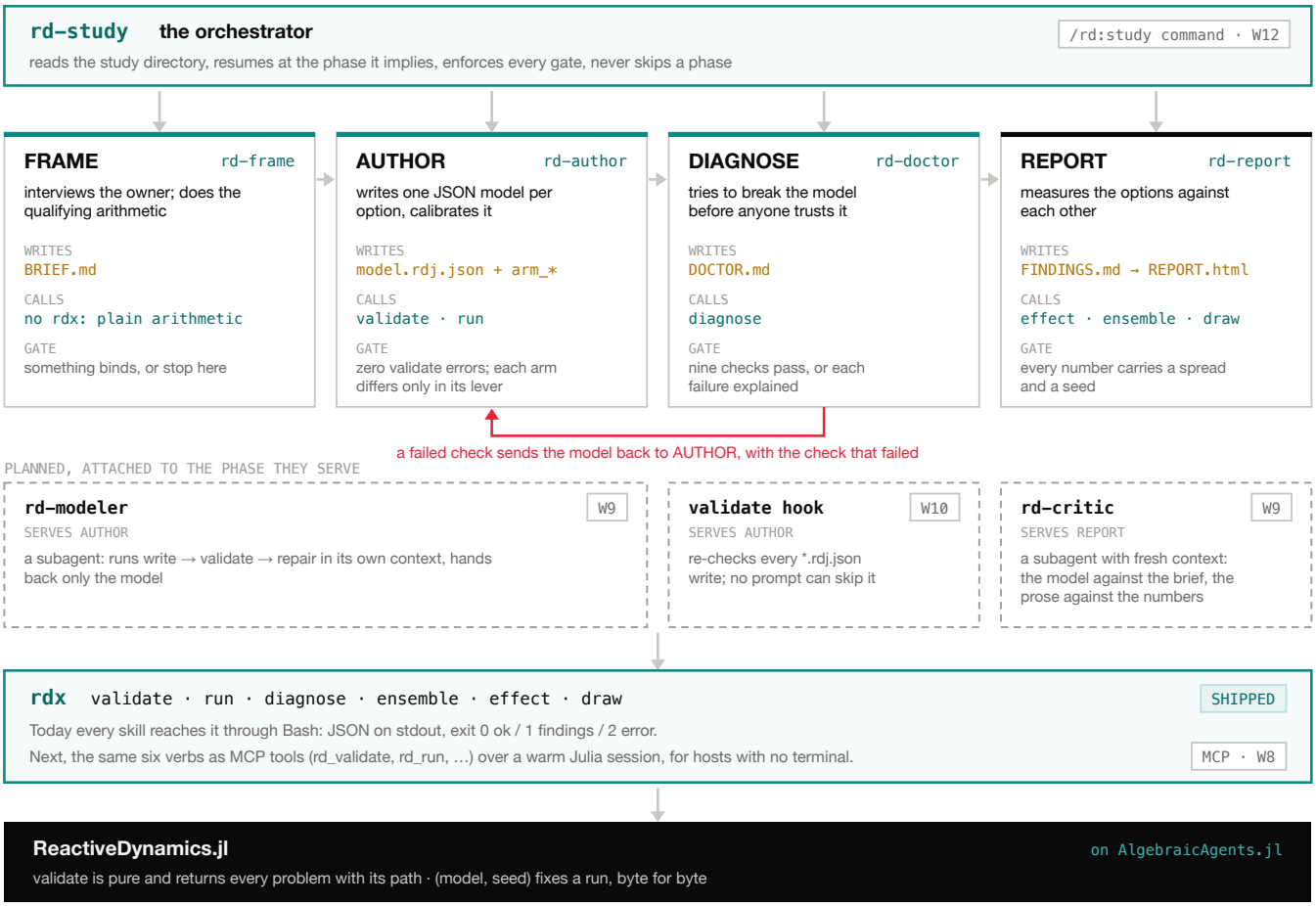


Solid boxes ship today; outlined boxes are the direction, each with its tracker id.

The division follows what each kind of component is good for. A **skill** is procedure the assistant loads only when the phase needs it, so the conversation carries one phase's rules at a time. A **subagent** buys a clean context: the author's repair loop produces pages of diagnostics nobody needs to see, and a critic is only useful if it has not read the author's reasoning. An **MCP server** buys reach, the same verbs for a host with no terminal, and a warm Julia session. A **hook** buys enforcement a prompt cannot promise, because it runs whether or not the assistant remembers to.

Inside the copilot: who does what, and what passes between them

One orchestrator, four phase skills that each leave a file, and one CLI underneath. Work passes between the phases only as files on disk, which is what makes each one re-runnable and auditable.



How the parts hand work to each other. Phases pass work only as files on disk, and the one backward edge is DIAGNOSE returning a failed model to AUTHOR.

Every component, its role and what it works with:

Component	Kind	What it does	Works with	Status
`rd-study`	skill, orchestrator	reads the study directory, resumes at the phase it implies, enforces the gates	the user; the four phase skills	shipped
`rd-frame`	skill	interviews the owner, runs the qualifying arithmetic, pre-registers what would change the answer; writes `BRIEF.md`	the user	shipped
`rd-author`	skill	turns the brief into one model document per option and repairs it until `validate` is silent	`rdx validate`, `rdx run`; the generated schema reference	shipped
`rd-doctor`	skill	runs the nine plausibility checks and writes `DOCTOR.md`; a failure goes back to AUTHOR	`rdx diagnose`	shipped
`rd-report`	skill	measures the options on paired paths or ensembles and writes `FINDINGS.md`, then the generated `REPORT.html`	`rdx effect`, `rdx ensemble`, `rdx draw`; `render_report.jl`	shipped

`rdx`	CLI	the six verbs over the engine; JSON out, exit 0 ok / 1 findings / 2 error	ReactiveDynamics.jl	shipped
knowledge	files	the normative spec and ADRs, cited by section; the schema reference, generated from the engine and drift-gated in CI	every skill reads it when needed	shipped
`rd-modeler`	subagent	runs `rd-author`'s procedure in its own context, so the repair loop's noise stays out of the conversation, and hands back only the validated model	`rdx`, through Bash or MCP	W9
`rd-critic`	subagent	fresh-context review: the model against the brief, the briefing's prose against its numbers	the study's files, read-only	W9
`rd`	MCP server	the six verbs as tools (`rd_validate`, `rd_run`, …) and the spec as resources, over a warm session	any MCP host	W8
validate hook	hook	validates every write to a `*.rdj.json` and returns the diagnostics to the assistant	`rdx validate`	W10
plugin	packaging	`/rd:study` and the phase commands, one-step install	Claude Code	W12

Section 8 shows the other half of the stack: what the engine receives from all this, traced from the conversation to the model document.

6 · WHAT EXISTS TODAY, AND WHERE IT IS GOING

Today. The engine, the four-phase workflow as five skills, the `rdx` tool, the generated schema reference with its drift gate, the known-answer test, the generated study report, and four published studies, three with their captured sessions. A study runs in Claude Code on a checkout of the repository: the `rd-study` skill runs the interview, writes the brief, authors the model, runs the checks and asks for sign-off at each gate, and every command and its result is visible in the conversation.

The direction, in the order it retires risk — first what makes the number more trustworthy, then what widens who can reach it:

Direction	What it changes	Tracker
Fast ensembles	an option-level ensemble in minutes rather than 53 minutes, so studies stop falling back to paired paths	W24
Declared dead steps	an option declares what it intentionally never fires, so a failed check means a bug	W20
Weighting in the engine	forced outcomes and odds-weighting inside `rdx effect`, not in session scripts	W27
A critic	a separate agent audits the briefing's prose against the numbers — the class of error this case retracted out loud	W9

Beyond contention	a qualifying test and checks for feedback and mid-course decisions; coupled models as documents	W25
Validate hook	every edit to a model document validated automatically	W10
MCP server	the same operations as tools any assistant can call, with no terminal	W8
Plugin and study card	one-step install; a one-page summary over the generated report	W12 · W11

None of this changes what a study is. The first half makes the answer harder to get wrong; the second half moves the copilot from something a developer sets up to something an analyst opens.

7 · THE CASE: DO WE PAY FOR CAPACITY BEFORE THE READOUTS?

The question. A VP of launch readiness has two late-stage biologics reading out nine months apart (Kestrel, 60% odds; Tern, 45%), one sterile fill-finish line that can carry one launch but maybe not two, a contract-manufacturer (CMO) offer that expires on 30 November 2026, and a CFO who says not to pay for capacity on a 45% asset. Reserve the CMO slot now, build a second line, or wait? The operator arrived leaning toward a reservation sized for Tern.

How it was captured. Both sides were AI. The operator was a role-played agent that wrote its facts to a sheet before the first message and answered only from it; the assistant started from an empty context and could not see the sheet. The sheet is published next to the transcript with an audit of what the interview missed.

What the copilot did. In three rounds of questions it recovered the constraint the operator said "people forget": a fill site must run three validation batches before the filing, or wait six months for a supplement. It built one model with a version per option and checked that the operator's figures went in right: it reproduces their ~$250M of Kestrel first-year revenue and 54 batches a year at peak. Both follow from the inputs by construction, so this is a consistency check, not evidence that the model is right; that is what the plausibility checks are for. It then compared the options on paired paths, with readout outcomes forced and weighted by the stated odds.

What it found.

- **Reserve the slot now, usable by either product, and review it at Kestrel's readout: +$3.8M ± $1.1M against waiting** at the portfolio committee's odds, and +$25M if a batch earns its peak-year value.
- **At the operator's own, lower odds for Kestrel (50%), it is break-even**: +$1.6M ± $0.9M, just failing the brief's test. The recommendation is supply protection, not a bet on the odds.

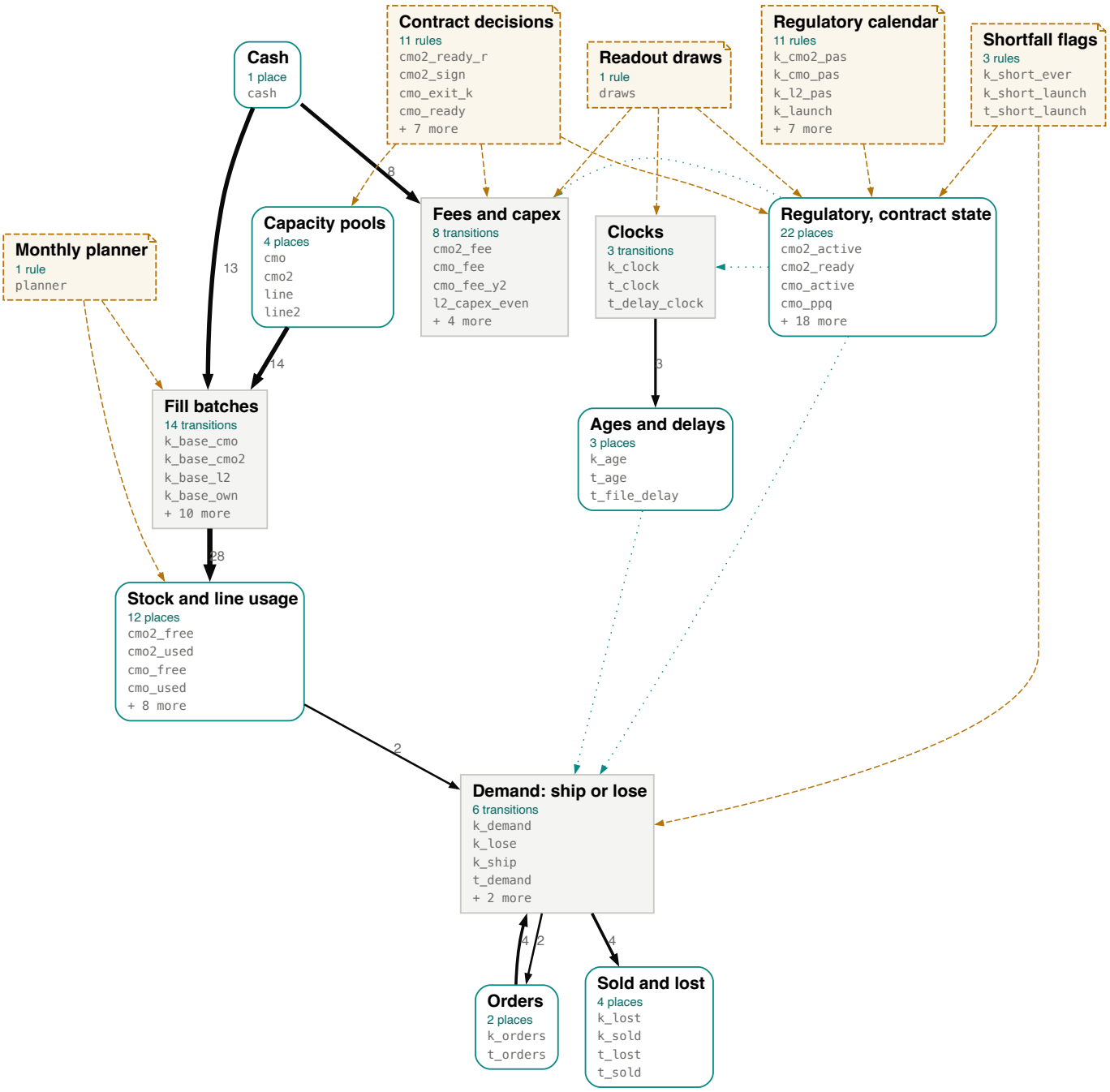
- **If both succeed (27%), the reservation is worth about +$39M.** Waiting leaves Tern short at launch on 26 of 37 paths and loses Tern demand on all 37, on average 11 batches worth $51M. In every other outcome the reservation is a bounded $9–11M cost.
- **A second line is $73–105M worse than waiting** even when both succeed, confirming the operator's view.
- **A Tern-only reservation is safe only with a second CMO contract in writing.** Without one it lost $72.5M when both succeed (3 paths, indicative), and the operator took it off the table.

The moment worth watching. The first pass came out at roughly break-even, because a rigid reservation carried a $72M loss on one path where only Tern succeeds. The model made that tail visible, and the operator supplied the fix: Kestrel reads out nine months earlier, so drop the reservation if Kestrel fails. The copilot encoded the rule, re-ran 30 paired paths and reported the result. The excerpt at the end of this page is that exchange, with the full session in a fold after it; it is also published as `agent/sessions/launch_supply.html`.

What the modeler sends on is the generated report, `agent/studies/launch_supply/REPORT.pdf` (18 pages): the answer and decision rule, the evidence, why a spreadsheet cannot produce it, the model in the operator's words with the one change that is the option, the execution map, and provenance down to the commit and all 208 stored runs. It is built from the study's files by `agent/scripts/render_report.jl` and never edited by hand.

8 · FROM THE CONVERSATION TO THE MODEL

The option the operator took to ExCo, *reserve now and review at Kestrel's readout*, is one JSON document: `arm_cmo_exit.rdj.json`. Because it is data, not code, every sentence the operator said can be traced to the element it became. The map below shows the whole model at the level of its parts, and the fold under it opens the same model element by element, drawn from the engine's own view of the net. The rows after it follow one quote at a time: what the operator said, the element as the document holds it, and what that element does. The document stores each expression as a tree of typed nodes; the rows print those trees as formulas so they can be read. In this study the documents were emitted by a generator script (`gen_models.py`) and each was then validated as data, so what is shown here is the validated document, not the script.



The model behind the recommendation, one box per part. Generated from `arm_cmo_exit.rdj.json`: all 48 places, 31 transitions and 27 rules, each in exactly one box. Solid arrows are the model's arcs (the number is how many), dotted arrows are rates that read the live state, dashed arrows are rules setting a marking, or a parameter that a rate reads.

In the screen version of this page: The same model, every element: 48 places, 31 transitions and 27 rules.

> *Operator, turn 3:* “Effective is about 13 after changeovers and two 2-week shutdowns a year; the last 12 months averaged 12.8.”

```
place line · starts at 128
```

```
transition k_base_own
  takes    line [conserved] + cash×15
  gives    k_stock + own_used
  rate     r_k_base_own
  holds    1 month(s)
  succeeds 0.97
  priority 100
```

The line is a pool of 128 tenths of a batch per month: the measured 12.8, not the nominal 16. Every batch filled on it, here Kestrel's launch stock, takes one tenth for the month it runs and gives it back after (`conserved`). That is why Kestrel, Tern and the marketed products can collide on it. The `cash×15` is the $150k variable cost per batch, charged in $1k units.

> *Operator, turn 3:* “the fill site has to be in the BLA with three PPQ batches before we file”

```
rule t_file  (once)
  when  (((t_ok > 0.5) && (t >= ((14 + t_shift) + 3))) && !(t_filed > 0.5)) && (((cmo_now > 0.5) && (cmo_ppq >
0.5)) || (!(cmo_now > 0.5) && (t_ppq_done >= 30)))
  then  t_filed := 1
        t_file_t := t

rule t_cmo_pas  (once)
  when  ((!(cmo_now > 0.5) && (cmo_ppq > 0.5)) && (t_filed > 0.5)) && (t >= (max(cmo_ready_t, (t_file_t + 12))
+ 6))
  then  t_cmo_ok := 1
```

The constraint the operator said people forget became a guard. Tern files only when its readout succeeded, three months have passed, and a fill site has its three PPQ batches, either at the CMO or on the own line (`t_ppq_done >= 30` tenths). If the CMO qualifies after the filing, `t_cmo_pas` adds it six months later, the Prior Approval Supplement from the same answer.

> *Operator, turn 14:* “I'd move Tern's PPQ in-house, file Tern on our own line, and give the CMO notice.”

```
rule cmo_exit_k  (once)
  when  (((cmo_now > 0.5) && (exit_at_k > 0.5)) && !(k_ok > 0.5)) && (t >= (5 + k_slip))
  then  cmo_active := 0
        cmo := 0
        cmo_ppq := 0
        cmo_now := 0
        cmo_exited := 1

param exit_at_k = 1    (the base model has 0: this is the option)

transition cmo_fee_y2
  takes    cash×5000
```

```
  gives    nothing (a pure cost)
  rate     1 * ((((cmo_now > 0.5) || (cmo_exited > 0.5)) && (t == 14)) && ((!(!(t_ok > 0.5) && (!(cmo_reassign
> 0.5) || !(k_ok > 0.5))) && !(cmo_exited > 0.5)) || (walk2 > 0.5)))
  priority 1
```

The operator's rule, added in turn 15, is one rule and one switch. At Kestrel's readout (month 5 plus the drawn slip), if Kestrel failed, the reservation is dropped and Tern's PPQ falls back to the own line through the `t_file` guard above. The forfeited year's fee is still charged. `exit_at_k` is the only line that differs from the rigid reservation, `arm_cmo.rdj.json`. That is what the inertness check tested: on the same seeds, the two options came out identical in every world where Tern does not succeed alone.

In the screen version of this page: 7 more: what else the operator said, and where it went.

Generated by `julia --project=. agent/scripts/render_mirror.jl agent/studies/launch_supply/mirror.json`. Every quote is checked verbatim against the transcript, and every element is printed from the model document.

In the screen version of this page: the option the operator chose, as the engine receives it: reserve now, review at Kestrel's readout (`agent/studies/launch_supply/arm_cmo_exit.rdj.json`).

9 · WHAT THIS DOES NOT SHOW

- **That a real person answers like the role-played one.** Both sides of the case were AI, and the operator was cooperative. It demonstrates the workflow, not anyone's supply plan.
- **Ensemble-grade evidence.** Every comparison rests on paired single-seed paths, because a 96-seed ensemble took 53 minutes (W24).
- **That the soft inputs were tested.** The interview recorded demand ±30%, marketed growth and readout correlation, then tested none of them. The published audit leads with them.
- **That the briefing's prose is checked.** The numbers are generated from runs; the prose is written by the model. The assistant caught and retracted two of its own claims in this case; a critic that does this systematically is W9.
- **That every pattern is checked equally.** Contention is checked end to end; feedback and mid-course decisions ran here without loop checks; coupled models do not yet go through the copilot (W25).
- **That the planned pieces exist.** The outlined boxes in §5 are the direction, not shipped.

13 EXCHANGES OMITTED Thirteen earlier turns: framing (the odds, the validation-batch rule, line capacity, the CMO terms), the model and its checks, a first report, and the operator's challenge, after which the assistant retracted two of its own claims and found the $72M loss on one Tern-only path.

THE OPERATOR CHANGES THE RULE

OPERATOR

This is what I can use. The corrections on "on time" and the worst case are exactly what I'd have been caught on. I won't quote late-filing numbers at all until you've resolved the mismatch with the baseline.

But your Tern-only result doesn't match how I'd actually run the business. Kestrel reads out in March 2027, nine months before Tern. If Kestrel fails, our line suddenly has plenty of room. I wouldn't keep paying $0.4M a batch at the CMO and bet Tern's filing on their qualification schedule. I'd move Tern's PPQ in-house, file Tern on our own line, and give the CMO notice. That means eating the $5–10M. So the reservation isn't a fixed bet on Tern; it's something we decide to keep or drop at Kestrel's readout. Model it that way:

- **Kestrel fails:** Tern's PPQ goes in-house, we give CMO notice, and we pay the forfeit.
- **Kestrel succeeds:** Tern stays at the CMO, as you have it now.

My guess is that this cuts the Tern-only loss down to roughly the both-fail cost, and takes the $72M tail away. If I'm wrong, show me.

On your questions:

1. **Backup qualification.** Regulatory will tell me it's feasible to list both sites in the BLA if both have PPQ data at filing. My estimate is about $0.5M a batch all-in including drug substance, so around $1.5M for three batches. The real cost, though, is three batches of line time right in the middle of Kestrel's stock build, which is the same collision you already found. So the backup only makes sense when Kestrel has failed. That's the rule above. Don't model the backup as a separate option in the both-succeed world unless it's cheap to do.

1. **Run budget.** Spend it on the Tern-only outcome, with the exit-at-Kestrel-readout rule built in. The no-second-contract sensitivity can wait, because I'm taking the "Tern-only" structure off the table unless we have the second contract in writing.

If the rule change brings the Tern-only loss into line, I'll take this version to ExCo: reserve now, review at Kestrel's readout. Send me that result and the final one-paragraph wording, and we're done.

CLAUDE

Phase 4 of 4, the final result. Your guess holds: under the review rule, the Tern-only outcome costs $9.4M against waiting, and the $72M tail is gone.

The rule: reserve the CMO now and review at Kestrel's readout. If Kestrel fails, Tern's validation batches move in-house, Tern files on our own line, we give the CMO notice and pay the forfeit. If Kestrel succeeds, nothing changes.

- **Tern-only outcome, 30 paired paths:** –$9.4M ± $0.02M against waiting. Every path falls between –$9.7M and –$9.3M.

- It is pure cost. Tern's revenue is the same as under waiting on all 30 paths, Tern files on time on all 30, and no Tern demand is lost on any of them.
- The old rule (keep the CMO no matter what) lost $31.7M ± $6.0M in this outcome over 10 paths. Four of those 10 lost more than $20M, the worst lost $72.1M, and 4 of 10 filed late.
- **Every outcome except both-succeed now costs about the same:**

Outcome	Cost vs waiting	Paths
Kestrel only	$8.8M	3
Tern only	$9.4M	30
Both fail	$9.4M	3

The 3-path outcomes vary by less than $0.1M across their paths.

- **Expected value:** +$3.8M ± $1.1M against waiting (plausible range +$1.6M to +$6.0M). This now passes the brief's test that the value minus two standard errors stays above zero. Last turn it failed, at –$1.1M ± $3.4M.
- At the peak value per batch: +$25.0M ± $2.8M.
- If Kestrel's odds of success are 0.50: +$1.6M ± $0.9M, which just misses the test (–$0.2M at the low end).
- **What the review rule does not change:** when both succeed, the rule never fires. The value there stays at +$38.7M ± $4.1M over 37 paths, and Tern's launch is fully supplied on all 37 (under waiting, Tern is short on 26 of 37). The remaining downside is also in that outcome: 3 of 37 paths lose money, the worst $14.3M. That happens when Tern reads out before the CMO has qualified and has to wait for it.

The late-filing mismatch is resolved. It was sampling chance, not a model error.

- I re-ran three members of the 96-seed baseline with their exact seeds. They reproduce its late-filing months and lost Tern demand exactly, with or without the outcomes forced.
- The value gap is a constant 0.72% from a one-month discounting convention, which cancels out of every comparison.
- Our 13 late filings in 37 paths against the baseline's 2 in 20 fits chance ($p = 0.06$). Pooled, the rate is 15 in 57 (range 0.17–0.39). The two sources do differ on how long a late filing runs: 2–3 months in our paths, 2 months in both of the baseline's cases.
- Late filing is about as likely either way: 0.10 overall under reserve-and-review, 0.095 under waiting.
- The figures are usable now, but quote them with their range: they are the least precise numbers in the report.

Backup qualification: I did not model it at your $0.5M a batch. The model charges $0.15M a batch for in-house validation, so its cost in the Tern-only outcome is understated by about $1M. That is arithmetic, not a simulated result.

One paragraph for ExCo:

> We recommend reserving the CMO fill-finish slot now, before 30 November 2026, usable by either Kestrel or Tern, and reviewing it at Kestrel's readout in March 2027. If Kestrel fails, Tern's validation batches move to our own line, Tern files there, and we give the CMO notice and pay the forfeit. If Kestrel succeeds, the reservation stands. Modelled over paired simulated paths, this is worth about +$3.8M (±$1.1M) against waiting at the stated odds, and about +$25M if a batch earns its peak-year value. Its purpose is supply protection. If both assets succeed (27% chance), it is worth about +$39M and removes a Tern launch shortfall that otherwise occurs on

> 26 of 37 paths, costing on average about 11 batches and $51M. If either asset fails, the cost is a bounded $9–10M: over 30 paths in the Tern-only case, and 3 each for Kestrel-only and both-fail. We do not recommend building a second line: it is about $84–87M worse than waiting even when both succeed (8 paths). We do not recommend a Tern-only reservation unless the second CMO contract is in writing. The one remaining risk is a late CMO qualification delaying Tern's filing if both succeed: 3 of 37 paths lose money, the worst about $14M.

▸ `MODEL` **arm: reserve now, review at Kestrel's readout (the operator's rule)** 48 places · 31 transitions · 78 arcs · 27 rules

the whole difference from `model.rdj.json`

```
--- model.rdj.json
+++ arm_cmo_exit.rdj.json
@@ -88,7 +88,7 @@
    },
    {
     "name": "cmo_now",
-    "value": 0.0
+    "value": 1.0
    },
    {
     "name": "cmo_reassign",
@@ -128,7 +128,7 @@
    },
    {
     "name": "exit_at_k",
-    "value": 0.0
+    "value": 1.0
    },
    {
     "name": "cmo_exited",
```

▸ `RUN` 4 commands

```
$ python3 agent/studies/launch_supply/gen_models.py   # + arm_cmo_exit: at Kestrel readout, if Kestrel failed, move Tern PPQ in-house, give notice, pay forfeit  exit 0 · ok
  → wrote 23 documents + 48 scenario copies
  rdx validate arm_cmo_exit.rdj.json: ok, 0 findings
  regeneration check: stored runs arm_cmo both s1, model tonly s2 re-run byte-identical

$ run_scenarios.sh arm_cmo_exit both,konly,neither 1 · arm_cmo_exit tonly 1..30 · model tonly 4..30 · arm_cmo tonly 4..10 · 6 ensemble replays (members 20, 96, 6; forced and unforced)  exit 0 · ok
  → 75 runs this turn, every one exit 0

$ python3 agent/studies/launch_supply/exit_review.py  exit 0 · ok
  → inertness check, arm_cmo_exit vs arm_cmo on the same seed (must be 0): both 0.0 · konly 0.0 · neither 0.0
  TERN-ONLY WORLD, paired against wait
   arm_cmo_exit vs wait: mean -9.40 se 0.02 n 30; min -9.66 max -9.25 | Tern files late 0/30; Tern demand lost on 0/30
   arm_cmo      vs wait: mean -31.67 se 5.95 n 10; min -72.09 max -18.32 | Tern files late 4/10
  WEIGHTED 'reserve now, review at Kestrel readout' vs wait
    both +38.71 (n 37) | konly -8.76 (n 3) | tonly -9.40 (n 30) | neither -9.38 (n 3)
    weighted delta +3.81, se 1.10; -2se +1.62, +2se +6.00

$ python3 agent/studies/launch_supply/ens_check.py   # replay 3 members of the 96-seed wait ensemble on their exact seeds  exit 0 · ok
```

```
→ member 20: forced == unforced True; delay 2 t_lost 92 (ensemble 2 / 92); eNPV ratio 1.00721 (1.09^(1/12) =
1.00721)
member 96: forced == unforced True; delay 2 t_lost 59 (ensemble 2 / 59); ratio 1.00724
member 6:  forced == unforced True; delay 0 t_lost 91 (ensemble 0 / 91); ratio 1.00723
late filing, wait, both succeed: ours 13/37, ensemble 2/20; Fisher exact two-sided p = 0.059
```

`agent/studies/launch_supply/exit_review.py` `agent/studies/launch_supply/ens_check.py` `agent/studies/launch_supply/FINDINGS.md` `(revision 3)`

2 EXCHANGES OMITTED

The full session, all 17 turns with every command and its output, is in the screen version of this page and at `agent/sessions/launch_supply.html`.

Companion pages: `agent/copilot_deck.html` (this argument as slides), `agent/sessions/launch_supply.html` (the full transcript), `agent/sessions/launch_supply_facts.md` (the operator's sealed fact sheet and the audit), `agent/studies/launch_supply/REPORT.pdf` (the shareable report) and `agent/overview.html` (the component division and the `portfolio_bid` session). The engine is ReactiveDynamics.jl.